

Car Game

멀티플레이, 랭킹 기능 추가

202513069 양유빈

목차

1. 개요

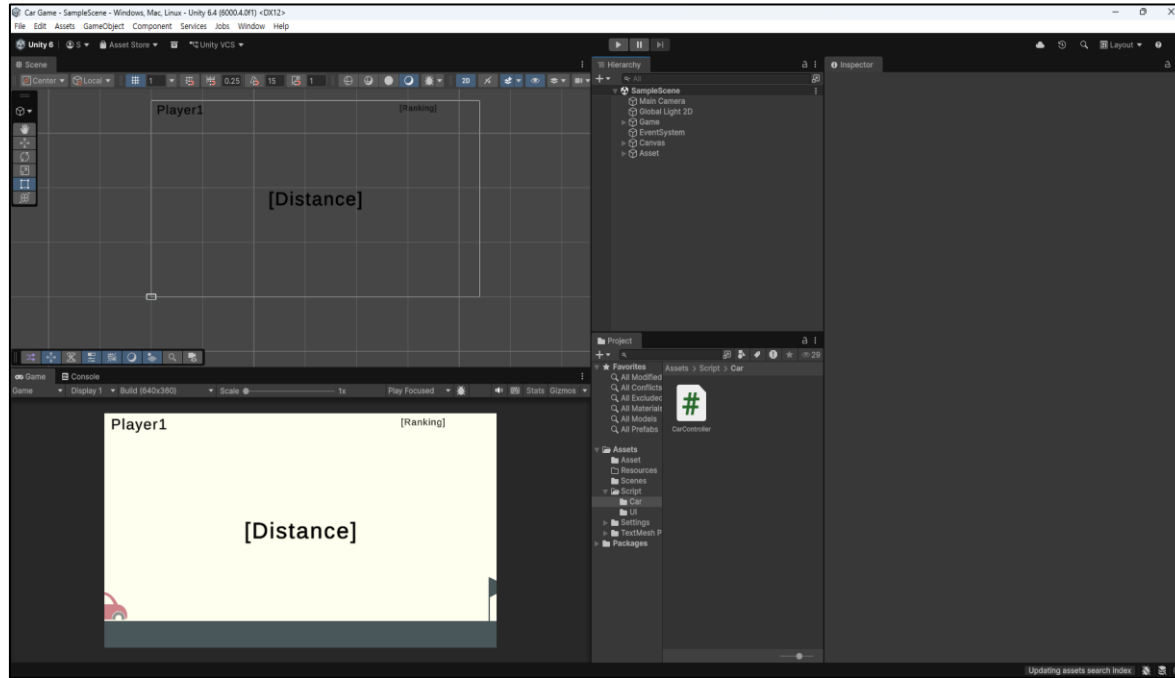
2. 시스템

구조, 서버와 클라이언트, 데이터베이스

3. 마무리

실행 결과, 배운 점

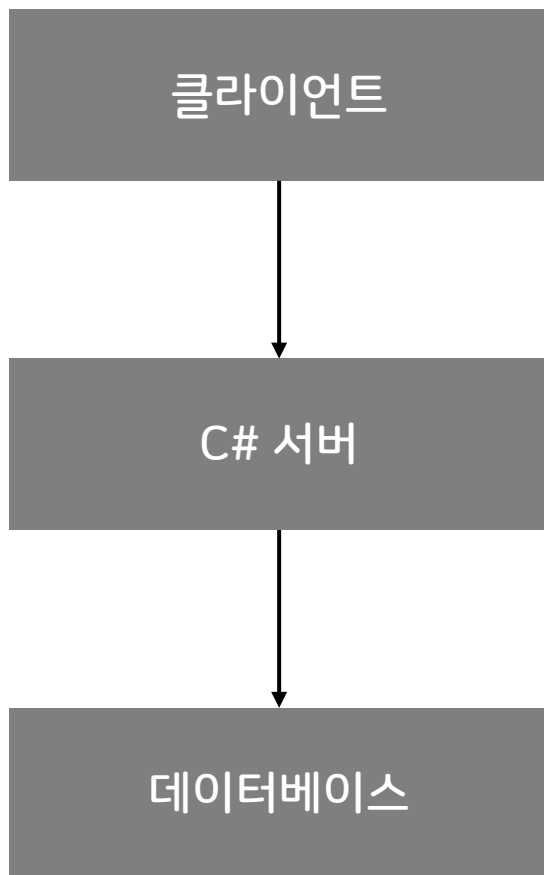
개요



<유니티 에디터 화면>

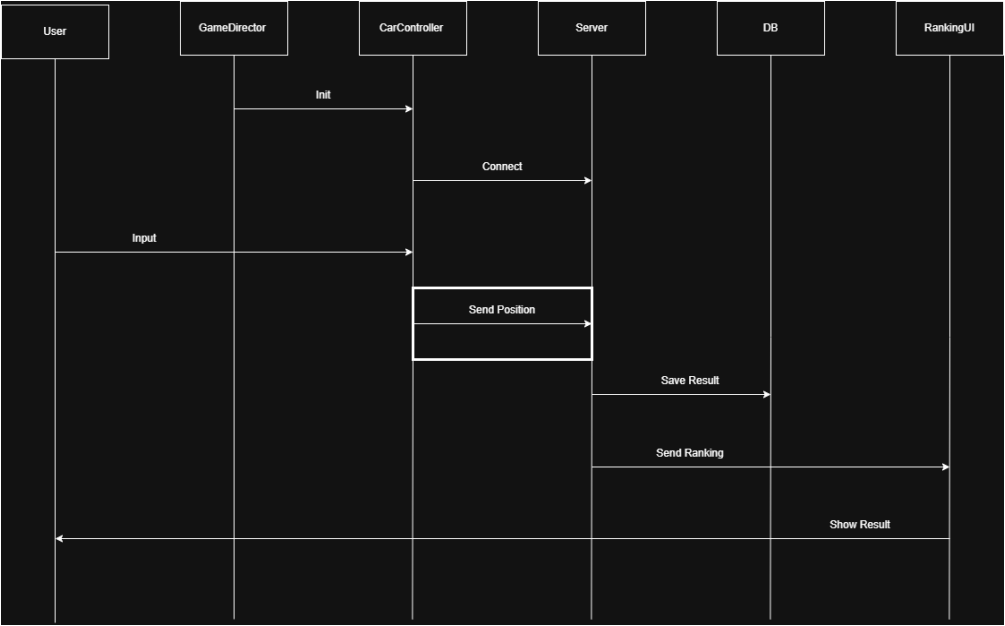
프로젝트 소개

- UDP 기반 2인용 게임이다.
- 두 플레이어의 위치를 실시간으로 동기화 할 수 있다.
- 승패 판정 후 기록을 데이터베이스에 저장할 수 있다.



프로젝트 구조

- 클라이언트는 위치 데이터를 서버로 전송한다.
- 서버에서 승패를 판정한 후에 기록을 데이터베이스에 저장한다.
- 서버에서 랭킹 테이블을 읽어 클라이언트에 전달한다.



<시퀀스 다이어그램>

시퀀스 다이어그램

- 1. 게임 시작 시 초기화
- 2. 서버 연결
- 3. 사용자 입력 발생
- 4. 위치 데이터 전송 (반복)
- 5. 게임 종료 후 결과 저장
- 6. 랭킹 데이터 전달
- 7. 사용자에게 결과 출력

Program.cs

<서버>

서버 (C#)

- UDP 수신, 위치 전달, 게임 종료 판정, DB 접근 기능이 있다.

```
참조 1개
static void ServerFunc(object obj)
{
    // UDP
    Socket srvSocket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.Udp);

    srvSocket.Bind(new IPEndPoint(IPAddress.Any, 12345));

    byte[] rcvBytes = new byte[1024];

    while (true)
    {
       EndPoint clientEP = new IPEndPoint(IPAddress.Any, 0);

        // 데이터 수신
        int nRecv = srvSocket.ReceiveFrom(rcvBytes, ref clientEP);

        string txt = Encoding.UTF8.GetString(rcvBytes, 0, nRecv);

        Console.WriteLine("[Recv] " + txt);

        string[] parts = txt.Split(' ');

        // 위치 동기화 처리
        if (parts.Length == 2)
        {
            string player = parts[0];

            if (player == "Player1")
                player1EP = clientEP;

            if (player == "Player2")
                player2EP = clientEP;

            byte[] sendBytes = Encoding.UTF8.GetBytes(txt);

            if (player == "Player1" && player2EP != null)
                srvSocket.SendTo(sendBytes, player2EP);

            if (player == "Player2" && player1EP != null)
                srvSocket.SendTo(sendBytes, player1EP);
        }

        // 게임 종료
        if (parts.Length == 3 && parts[2] == "END")
    }
}
```

<서버 소스 코드>

서버 (C#)

- 유니티와 별개의 솔루션 파일로 관리한다.
- 게임 시작 전에 서버를 실행시켜 줘야 한다.
- UDP를 기반으로 구현했다.
- 데이터 송수신 (위치 동기화, 승패 판정)을 담당한다.

CarController.cs

GameDirector.cs

RankingUI.cs

<클라이언트>

클라이언트 (유니티)

- 이동 및 UDP 송수신, 승패 표시, 랭킹 UI 출력 기능이 있다.


```
// 결과 저장 함수
참조 2개
static void SaveResult(string player, float dist, string result)
{
    string connStr =
        "Server=127.0.0.1;Port=3307;Database=ckgame;Uid=root;Pwd=000000;SslMode=None;";

    using (MySqlConnection conn = new MySqlConnection(connStr))
    {
        conn.Open();

        string sql =
            $"INSERT INTO Ranking VALUES('{player}', {dist}, '{result}')";

        MySqlCommand cmd = new MySqlCommand(sql, conn);

        cmd.ExecuteNonQuery();
    }
}

// Top5 함수
참조 1개
static string LoadTop5()
{
    string result = "TOP5";

    string connStr =
        "Server=127.0.0.1;Port=3307;Database=ckgame;Uid=root;Pwd=000000;SslMode=None;";

    using (MySqlConnection conn = new MySqlConnection(connStr))
    {
        conn.Open();

        string sql =
            "SELECT PlayerName, Distance FROM Ranking ORDER BY Distance ASC LIMIT 5";

        MySqlCommand cmd = new MySqlCommand(sql, conn);

        MySqlDataReader rdr = cmd.ExecuteReader();

        while (rdr.Read())
        {
            result += "|" +
                rdr.GetString(0) +
                ":" +
                rdr.GetFloat(1).ToString("F2");
        }
    }

    return result;
}
```

<데이터베이스 접근 소스 코드 (서버)>

클라이언트 (유니티)

- 클라이언트에서 데이터베이스에 접근할 수 없다.
- 따라서 서버에 있는 함수를 호출하는 방식으로 3-Teir를 구현했다.
- 경기 결과를 저장하고 랭킹을 불러오는 것이 가능하다.

```
mysql> use ckgame;
Database changed
mysql> desc ranking;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| PlayerName | varchar(20) | YES  |     | NULL    |      |
| Distance   | float      | YES  |     | NULL    |      |
| Result     | varchar(10) | YES  |     | NULL    |      |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> SELECT * FROM ranking;
+-----+-----+-----+
| PlayerName | Distance | Result |
+-----+-----+-----+
| Player1    | 4.6      | Win    |
| Player2    | 11.4     | Lose   |
| Player1    | 5.8      | Win    |
| Player2    | 8.8      | Lose   |
| Player1    | 7.6      | Lose   |
| Player2    | 5.9      | Win    |
| Player1    | 18.07    | Win    |
| Player2    | 18.23    | Lose   |
| Player1    | 4.9      | Lose   |
| Player2    | 4.4      | Win    |
| Player1    | 2.2      | Win    |
| Player2    | 19.6     | Lose   |
| Player1    | 11.2     | Lose   |
| Player2    | 3.5      | Win    |
+-----+-----+-----+
14 rows in set (0.01 sec)
```

<랭킹 테이블>

랭킹 테이블

- 플레이어 이름, 거리(기록), 게임 결과를 저장할 수 있다.
- 기록을 정렬하여 클라이언트에 전할 수 있다.
- 데이터베이스는 서버에서만 접근할 수 있다.

(1) 클라이언트와 서버 분리의 중요성

- 게임 랭킹과 기록 기능을 구현하면서 클라이언트에서 직접 모든 데이터를 처리하려고 했다.
- 하지만 프로그램이 복잡 해져서 클라이언트는 입력과 화면 표시를 담당하고 서버에서 데이터를 담당하는 것으로 분리했다.
- 두 가지 기능을 분리하는 것이 더 안정적이라는 것을 알 수 있었다.

(2) 서버와 데이터베이스 연동 이해

- 데이터베이스를 클라이언트에서 직접 접근하지 않고 서버를 통해 처리하도록 구성했다.
- 교수님이 3-Tier를 왜 강조 하셨는 지 알 수 있었다.
- 특히 게임 종료 후 서버에서 기록을 저장한 뒤 다시 랭킹 데이터를 전달하는 과정에서 3-Tier구조의 장점을 느낄 수 있었다.
- 클라이언트에서 데이터베이스에 접근하는 방법은 기록 저장 후 랭킹을 불러오는 과정에서 랭킹이 갱신되지 않는 문제가 있었다.